# 🔎 Querying Spans with DQL

> **Series:** SPANS | **Notebook:** 2 of 8 | **Created:** December 2025 | **Last Updated:** 01/28/2026

## Mastering Span Queries in Dynatrace

This notebook covers essential techniques for querying and filtering span data to find exactly what you need. You'll learn to filter by service, operation, and attributes to quickly locate relevant traces.

---

## Table of Contents



## Prerequisites

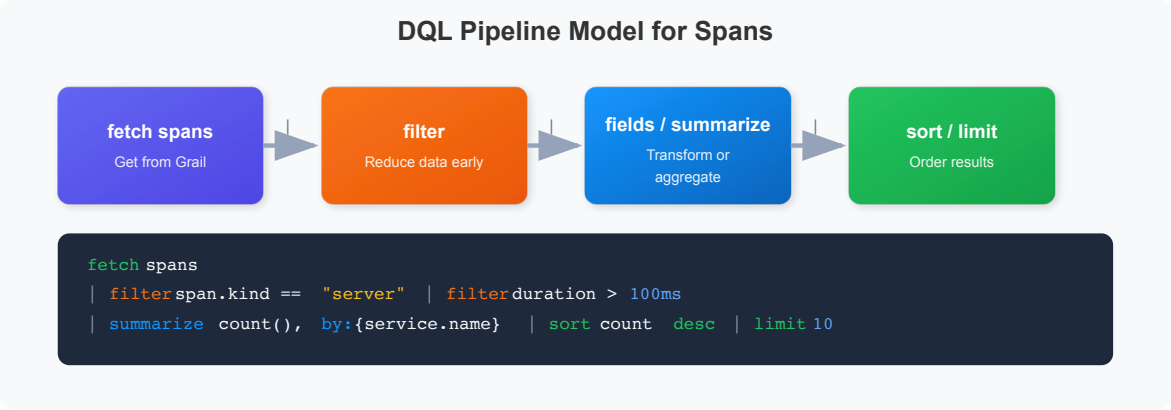
Before starting this notebook, ensure you have:

- ✅ Completed **SPANS-01: Fundamentals**

- ✅ Access to a Dynatrace environment with span data
- ✅ DQL query permissions

# 1. DQL is NOT SQL!

⚠️ **CRITICAL:** DQL has different syntax from SQL. Memorize these differences:




## Key Differences

| Concept | SQL | DQL |
|---|---|---|
| **Arrays** | `('a', 'b')` parentheses | `{"a", "b"}` curly braces |
| **Comparison** | `=` single equals | `==` double equals |
| **String quotes** | `'single quotes'` | `"double quotes"` |
| **NULL checks** | `IS NULL` / `IS NOT NULL` | `isNull()` / `isNotNull()` |
| **Membership** | `IN (...)` | `in(field, {...})` |
| **Grouping** | `GROUP BY` | `by: {...}` in summarize |

# 2. Filtering by Service

Filter spans to focus on specific services. Use `dt.entity.service` for reliable filtering (entity ID), or `service.name` for display name.

> 💡 **Tip:** `dt.entity.service` is always populated and indexed. `service.name` may not always be available.

```
// Filter spans for a specific service using exact match
fetch spans
| filter service.name == "checkout"
| fields start_time, span.name, span.kind, duration
| sort start_time desc
| limit 50
```

```
// Use in() with curly braces {} for multiple values (NOT parentheses!)
fetch spans
| filter in(service.name, {"checkout", "payment", "cart"})
| fields start_time, service.name, span.name, span.kind, duration
| sort start_time desc
| limit 100
```

```
// Count spans by service entity (more reliable)
fetch spans
| filter isNotNull(dt.entity.service)
| summarize {span_count = count()}, by: {dt.entity.service}
| sort span_count desc
| limit 20
```

---

## 3. Filtering by Span Kind

Filter spans based on their role in the distributed transaction.

⚠️ **IMPORTANT:** `span.kind` values are **lowercase**!

| Kind | Description | Use Case |
| --- | --- | --- |
| `"server"` | Handles incoming request | Find inbound API calls |
| `"client"` | Makes outgoing request | Find calls to dependencies |
| `"internal"` | Internal processing | Find business logic |
| `"producer"` | Sends async message | Find message publishers |
| `"consumer"` | Receives async message | Find message consumers |

```
// Find all SERVER spans (inbound requests)
// Note: "server" is lowercase, not "SERVER"
fetch spans
| filter span.kind == "server"
| fields start_time, service.name, span.name, duration
| sort duration desc
| limit 50
```

```
// Find CLIENT spans (outbound calls to dependencies)
fetch spans
| filter span.kind == "client"
| fields start_time, service.name, span.name, duration
| sort duration desc
| limit 50
```

```
// Count spans by kind to understand your traffic patterns
fetch spans
| summarize {span_count = count()}, by: {span.kind}
| sort span_count desc
```

---

## 4. Filtering by Operation Name

Find spans for specific operations or endpoints using the `span.name` attribute:

```
// Find spans for a specific operation/endpoint
fetch spans
| filter contains(span.name, "checkout")
| fields start_time, service.name, trace.id, span.name, duration,
span.status_code
| sort start_time desc
| limit 50
```

```
// Find all POST operations (write operations)
fetch spans
| filter startsWith(span.name, "POST")
| fields start_time, service.name, span.name, duration, span.status_code
| sort start_time desc
| limit 50
```

---

## 5. String Matching Functions

DQL provides several string matching functions:

| Function | Description | Example |
|---|---|---|
| `contains(field, "text")` | Substring match | `contains(span.name, "user")` |
| `startsWith(field, "text")` | Prefix match | `startsWith(span.name, "GET")` |
| `endsWith(field, "text")` | Suffix match | `endsWith(url.path, ".json")` |
| `matchesPhrase(field, "pattern")` | Wildcard pattern | `matchesPhrase(span.name, "GET /api/*")` |
| `in(field, {"a", "b"})` | Multiple values | `in(span.kind, {"server", "client"})` |

```
// Contains - partial match anywhere in the string
fetch spans
| filter contains(span.name, "Get")
| fields span.name, service.name
| dedup span.name
| limit 20
```

```
// startsWith and endsWith - prefix/suffix matching
fetch spans
| filter startsWith(span.name, "GET") or endsWith(span.name, "query")
| fields span.name
| dedup span.name
| limit 20
```

```
// matchesPhrase - wildcard pattern matching with *
fetch spans
| filter matchesPhrase(span.name, "GET /api/*")
| fields span.name
| dedup span.name
| limit 20
```

```
// Use dedup to see unique span names per service
fetch spans
| filter span.kind == "server"
| fields service.name, span.name
| dedup service.name, span.name
| sort service.name asc
| limit 50
```

---

## 6. Finding Specific Traces

Locate all spans belonging to a specific trace:

```
// First, find some trace IDs to work with
fetch spans
| filter span.kind == "server"
| fields start_time, trace.id, span.name, service.name
| sort start_time desc
| limit 10
```

```
// Find all spans for a specific trace ID
// Replace YOUR_TRACE_ID with an actual trace.id from above
fetch spans
// | filter trace.id == "YOUR_TRACE_ID"
| fields start_time, span.id, span.parent_id, span.name, service.name,
duration
| sort start_time asc
| limit 100
```

```
// Find root spans (entry points) - spans without a parent
fetch spans
| filter isNull(span.parent_id)
| fields start_time, trace.id, span.name, service.name, duration,
span.status_code
| sort start_time desc
| limit 50
```

---

## 7. HTTP Span Queries

Query HTTP-specific span attributes for API troubleshooting:

| Attribute | Description |
| --- | --- |
| `http.request.method` | HTTP method (GET, POST, etc.) |
| `http.response.status_code` | HTTP status code (200, 404, 500) |
| `http.route` | URL route pattern (use this, not url.path for aggregation) |
| `url.path` | Full URL path (may contain PII) |

```
// Query HTTP spans with response status codes
fetch spans
| filter isNotNull(http.response.status_code)
| fields start_time,
         service.name,
         http.request.method,
         http.route,
         http.response.status_code,
         duration
| sort start_time desc
| limit 100
```

```
// Find HTTP 5xx errors (server errors)
fetch spans
| filter http.response.status_code &gt;= 500
      and http.response.status_code &lt; 600
| fields start_time,
         service.name,
         http.request.method,
         http.route,
         http.response.status_code,
         span.status_message,
         duration
| sort start_time desc
| limit 50
```

```
// Summarize HTTP status codes by route
fetch spans
| filter isNotNull(http.response.status_code)
| summarize {status_count = count()}, by: {http.response.status_code}
| sort http.response.status_code asc
```

---

## 8. Database Span Queries

Analyze database operations captured as spans:

| Attribute | Description |
| --- | --- |
| `db.system` | Database type (mysql, postgresql, redis) |
| `db.name` | Database name |
| `db.operation` | Operation type (SELECT, INSERT, UPDATE) |
| `db.statement` | The database query (may contain sensitive data) |

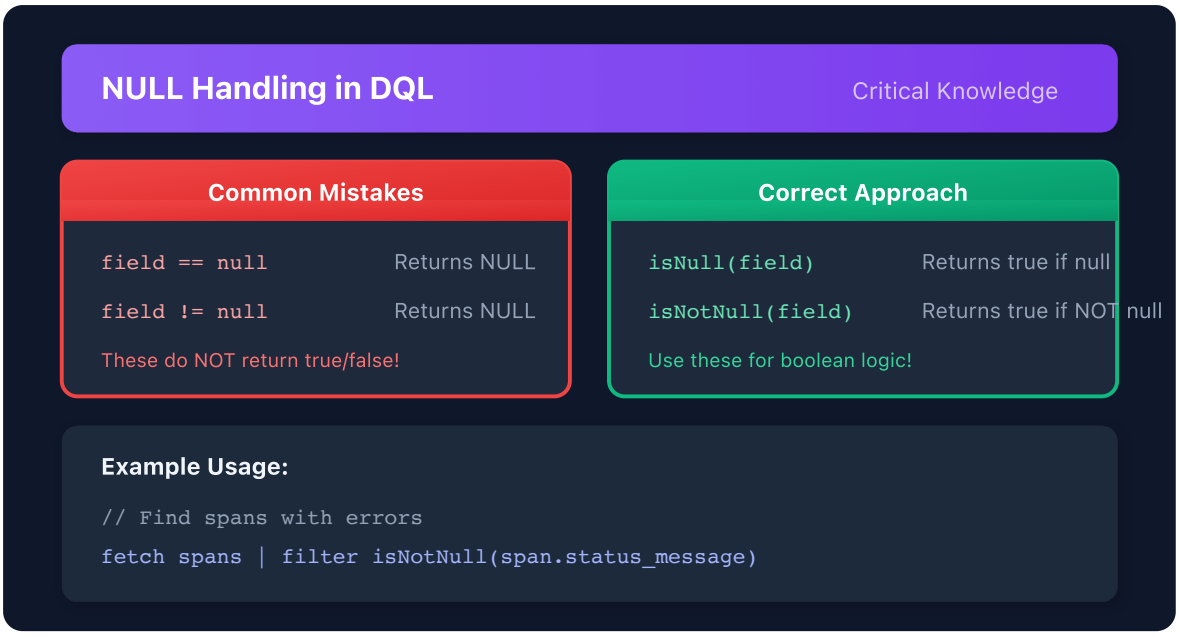
```
// Find all database spans
fetch spans
| filter isNotNull(db.system)
| fields start_time,
         service.name,
         db.system,
         db.name,
         db.operation,
         duration
| sort duration desc
| limit 50
```

```
// Find slow database queries (over 100ms)
fetch spans
| filter isNotNull(db.system)
      and duration &gt; 100ms
| fieldsAdd duration_ms = duration / 1000000
| fields start_time,
         service.name,
         db.system,
         db.operation,
         db.statement,
         duration_ms
| sort duration_ms desc
| limit 50
```

```
// Summarize database usage
fetch spans
| filter isNotNull(db.system)
| summarize {
    db_span_count = count(),
    avg_duration_ms = avg(duration) / 1000000
  }, by: {db.system, db.name}
| sort db_span_count desc
```

## 9. Working with NULL Values

⚠️ **DQL uses tri-state boolean logic.** Comparisons with NULL don't work like SQL!




```
// Find spans that have database information
fetch spans
| filter isNotNull(db.system)
| summarize {db_span_count = count()}, by: {db.system, db.name}
| sort db_span_count desc
```

```
// Find HTTP spans with missing route (potential instrumentation issue)
fetch spans
| filter isNotNull(http.request.method) and isNull(http.route)
| fields service.name, span.name, http.request.method, url.path
| dedup service.name, span.name
| limit 20
```

## 10. Combining Multiple Filters

Build complex queries by combining multiple filter conditions:

> 💡 **Performance Tip:** Apply more restrictive filters first for better query performance.

```
// Complex filter: Find slow SERVER spans in the checkout service
fetch spans
| filter service.name == "checkout"
      and span.kind == "server"
      and duration &gt; 500ms
| fieldsAdd duration_ms = duration / 1000000
| fields start_time,
         span.name,
         duration_ms,
         span.status_code
| sort duration_ms desc
| limit 50
```

```
// Find error spans for specific services and operations
fetch spans
| filter span.status_code == "error"
      and in(service.name, {"payment", "checkout"})
      and span.kind == "server"
| fieldsAdd duration_ms = duration / 1000000
| fields start_time,
         service.name,
         span.name,
         span.status_message,
         trace.id,
         duration_ms
| sort start_time desc
| limit 50
```

```
// Find spans: either server errors OR slow successful requests
fetch spans
| filter http.response.status_code &gt;= 500
      or (http.response.status_code &gt;= 200
          and http.response.status_code &lt; 300
          and duration &gt; 1s)
| fieldsAdd duration_ms = duration / 1000000
| fields start_time,
         service.name,
         http.request.method,
         http.route,
         http.response.status_code,
         duration_ms
| sort start_time desc
| limit 100
```

---

## Summary

In this notebook, you learned:

✅ **DQL ≠ SQL** - Critical syntax differences (arrays use `{}`, use `==`, `isNull()`)
✅ **Filter by service** using exact match, `in()`, and pattern matching
✅ **Filter by span kind** - values are lowercase (`"server"`, `"client"`)
✅ **String matching** - `contains()`, `startsWith()`, `endsWith()`, `matchesPhrase()`
✅ **Find traces** by trace.id and identify root spans
✅ **Query HTTP spans** including status codes, methods, routes
✅ **Analyze database spans** to find slow queries
✅ **Handle NULL values** with `isNull()` / `isNotNull()`
✅ **Use** `dedup` to see unique values
✅ **Combine filters** for complex, precise queries

---

## Next Steps

Continue to **SPANS-03: Trace Analysis & Troubleshooting** to learn:
- Identifying error patterns and failure points
- Latency analysis across services

- Root cause analysis techniques
- Tracing request flows through your system

---